Video 1.1: Container Architecture and Isolation from a Security Perspective

Duration: 8:00
Format: PPT (30%) / Terminal (45%) / Code Editor (15%) / Browser (10%)
Resolution: 1920x1080 @ 30fps
Prerequisites: Basic Linux CLI

SCRIPT BEGIN

[0:00–0:45] — Opening Context (PPT Slide)

SLIDE: Title: “What Is a Container, Really?” — Diagram showing a process with dotted-line boundaries around it labeled “namespace,” “cgroup,” “filesystem”

NARRATION:
“Before we secure containers, we need to understand what a container actually is — not the marketing version, but the kernel version. A container is not a lightweight virtual machine. A container is a regular Linux process — or group of processes — with three layers of restriction applied to it. First, namespaces limit what the process can see. Second, cgroups limit what the process can use. Third, a layered filesystem limits what the process can access on disk. That’s it. There is no container object in the Linux kernel. There’s no ‘container’ system call. It’s just a process with boundaries. And understanding those boundaries is the foundation of container security.”

[0:45–1:15] — Namespace Overview (PPT Slide)

SLIDE: Table showing Linux namespaces:

Namespace	Isolates	Kernel Flag
PID	Process IDs	CLONE_NEWPID
NET	Network stack	CLONE_NEWNET
MNT	Mount points	CLONE_NEWNS
UTS	Hostname	CLONE_NEWUTS
IPC	IPC resources	CLONE_NEWIPC
USER	User/Group IDs	CLONE_NEWUSER
Cgroup	Cgroup root	CLONE_NEWCGROUP
Time	System clocks	CLONE_NEWTIME

NARRATION:
“Linux provides eight types of namespaces, each isolating a different aspect of the system. PID namespaces give a process its own process ID tree — inside the namespace, it thinks it’s PID 1. NET namespaces give it a separate network stack with its own interfaces, routes, and firewall rules. Mount namespaces isolate the filesystem view. UTS isolates the hostname. IPC isolates inter-process communication. User namespaces remap user IDs. And there are cgroup and time namespaces as well. When Docker creates a container, it uses the `clone()` or `unshare()` system calls with combinations of these flags to build the isolation boundary.”

[1:15–3:00] — Hands-On: Building Isolation with unshare (Terminal)

SCREEN: Terminal at 150% zoom, dark theme, max 20 lines visible.

NARRATION:
“Let’s prove this. I’m going to create container-like isolation using nothing but the `unshare` command. No Docker.”

COMMAND + EXPLANATION:

```
# Step 1: Show current PID and hostname on the host
$ echo "Host PID: $$, Hostname: $(hostname)"
```

“First, let’s note our current PID and hostname on the host system.”

```
# Step 2: Create a new PID, UTS, and mount namespace
$ sudo unshare --pid --uts --mount --fork bash
```

“Now I’m calling unshare with PID, UTS, and mount namespace flags. The fork flag ensures the new bash process becomes PID 1 inside the new PID namespace.”

```
# Step 3: Verify isolation — inside the new namespace
# mount proc so ps works correctly in the new PID namespace
$ mount -t proc proc /proc
$ echo "Namespace PID: $$"
$ ps aux
```

“Look at that — our shell is PID 1. And `ps aux` only shows processes inside this namespace. The host’s hundreds of processes are invisible to us. This is PID namespace isolation.”

```
# Step 4: Change hostname — UTS namespace isolation
$ hostname container-demo
$ hostname
container-demo
```

“I can change the hostname and it only affects this namespace. The host’s hostname is untouched.”

```
# Step 5: Verify host is unaffected — open a SECOND terminal tab
# (On host)
$ hostname
original-hostname
$ ps aux | grep unshare
```

“In a second terminal on the host, our hostname is unchanged. And we can see the unshare process — because from the host’s perspective, the ‘container’ is just a process.”

```
# Step 6: Exit the namespace
$ exit
```

[3:00–4:15] — Inspecting Namespaces from the Host (Terminal)

NARRATION:
“Now let’s inspect namespaces the way a security analyst would — from the host side.”

COMMAND + EXPLANATION:

```
# Start a Docker container for comparison
$ docker run -d --name ns-demo alpine sleep 3600
```

“Let me start an Alpine container so we can inspect its namespaces.”

```
# Find the container's PID on the host
$ docker inspect --format '{{.State.Pid}}' ns-demo
# Let's say this returns 4521
```

```
# List the namespaces for this process
$ ls -la /proc/4521/ns/
```

“Every process on Linux has a `/proc/[pid]/ns/` directory that shows its namespace memberships. Each entry is a symbolic link to a namespace object. Two processes in the same namespace will have the same inode number.”

```
# Compare with PID 1 (init) on the host
$ ls -la /proc/1/ns/
```

“Compare these with PID 1 — the host’s init process. Different inode numbers mean different namespaces — that’s the isolation boundary.”

```
# Use lsns for a cleaner view
$ lsns -p 4521
```

“`lsns` gives you a cleaner view — showing each namespace type, its inode, the number of processes inside it, and the owning user. This is one of your primary tools for auditing container isolation.”

```
# Clean up
$ docker rm -f ns-demo
```

[4:15–5:30] — Cgroups: Resource Limits (PPT + Terminal)

SLIDE: Title: “Cgroups — Controlling Resource Consumption”
Diagram: Process inside a box with gauges for CPU, Memory, I/O, PIDs

NARRATION:
“Namespaces control what a process can see. Cgroups control what it can use. Control groups — cgroups — allow the kernel to limit CPU time, memory, disk I/O, network bandwidth, and even the number of processes a container can create.”

SWITCH TO TERMINAL:

```
# Run a container with resource limits
$ docker run -d --name cgroup-demo \
    --memory=128m \
    --cpus=0.5 \
    --pids-limit=50 \
    alpine sleep 3600
```

“Let’s start a container with explicit resource limits — 128 megabytes of memory, half a CPU core, and a maximum of 50 processes.”

```
# Find the cgroup for this container (cgroup v2)
$ CONTAINER_ID=$(docker inspect --format '{{.Id}}' cgroup-demo)
$ cat /sys/fs/cgroup/system.slice/docker-${CONTAINER_ID}.scope/memory.max
# Output: 134217728 (128MB in bytes)
```

“The kernel enforces these limits through the cgroup filesystem. Here you can see the memory limit — 134 million bytes, exactly 128 megabytes.”

```
# Check CPU limit
$ cat /sys/fs/cgroup/system.slice/docker-${CONTAINER_ID}.scope/cpu.max
# Output: 50000 100000 (50% of one core)
```

“The CPU limit shows 50,000 out of 100,000 microseconds per period — that’s our half-CPU limit.”

```
# Check PIDs limit
$ cat /sys/fs/cgroup/system.slice/docker-${CONTAINER_ID}.scope/pids.max
# Output: 50
```

“And the PID limit is set to 50. If a containerized process tries to fork-bomb, it’ll hit this ceiling.”

[5:30–6:15] — Security Implications of Cgroups (PPT Slide)

SLIDE: Title: “Why Cgroups Matter for Security” — Two columns:

Without Cgroup Limits:
- Single container can consume all host memory → OOM kills other containers
- Fork bomb inside container → exhausts host PID space
- I/O-heavy container → starves neighboring workloads
- CPU-intensive process → degrades all co-tenants

With Cgroup Limits:
- Memory bounded → container is killed before host impact
- PID limits → fork bomb contained
- I/O throttling → fair sharing enforced
- CPU quotas → predictable performance

NARRATION:
“From a security standpoint, cgroups are your defense against denial-of-service conditions. Without cgroup limits, a single rogue container can consume all host memory, fork-bomb the process table, or starve other workloads of I/O. With limits properly configured, the damage is contained. The container itself might crash, but the host and neighboring containers survive. This is the security principle of blast radius reduction — and cgroups are how you enforce it at the kernel level.”

[6:15–7:15] — Layered Filesystem Security (Code Editor + Terminal)

SCREEN: Code editor at 150% zoom showing a Dockerfile

NARRATION:
“The third pillar of container isolation is the layered filesystem — implemented through overlay filesystems.”

CODE EDITOR — Dockerfile:

```
FROM ubuntu:22.04
# Layer 1: Base OS

RUN apt-get update && apt-get install -y nginx
# Layer 2: Install nginx

COPY nginx.conf /etc/nginx/nginx.conf
# Layer 3: Configuration

RUN echo "sensitive_data" > /tmp/secret.txt && \
    rm /tmp/secret.txt
# Layer 4: SECURITY RISK — data persists in layer!
```

“Here’s a critical security concept. Each instruction in a Dockerfile creates an immutable layer. Even if you delete a file in a later instruction, the data still exists in the previous layer. This is a common source of credential leaks — developers add secrets, then remove them, thinking they’re gone. They’re not.”

SWITCH TO TERMINAL:

```
# Demonstrate layer persistence
$ docker build -t layer-demo .
$ docker history layer-demo
```

“Docker history shows each layer. And with tools like `dive` or manual layer extraction, an attacker can recover deleted secrets from any layer.”

```
# Extract and inspect layers
$ docker save layer-demo -o layer-demo.tar
$ mkdir layers && tar -xf layer-demo.tar -C layers/
$ find layers/ -name "secret.txt"
```

“There it is — the ‘deleted’ secret, still sitting in the layer archive. We’ll cover how to prevent this properly in later modules.”

[7:15–8:00] — Section Recap and Bridge (PPT Slide)

SLIDE: Title: “Key Takeaways” — Three pillars diagram:

1. **Namespaces** → Visibility isolation (what you can see)
2. **Cgroups** → Resource isolation (what you can use)
3. **Layered FS** → Filesystem isolation (what you can access)

“These are the building blocks. None of them are perfect — they’re not designed to be security boundaries in the way a hypervisor is. They’re process-level restrictions. And that distinction is critical, because in the next lesson, we’re going to compare this model directly to virtual machine isolation, and you’ll see exactly where containers fall short.”

SLIDE: “Next: Container vs. VM Security Models →”

SCRIPT END

Post-Production Notes

- Terminal recordings should use a clean dark theme (e.g., Dracula or One Dark)
- All commands must be pre-tested on Ubuntu 22.04 with Docker 24.x+
- For the namespace comparison, use split-screen showing host terminal and namespace terminal side by side
- The Dockerfile layer demo should highlight the security risk line in red
- Cgroup filesystem paths assume cgroup v2 — add a note overlay if the system uses cgroup v1
- Pause briefly (1 second) after each command output before narrating the explanation